

LAMPSys - An experiment-platform for the optimization of processes and agent-behaviors

Christoph Mies, Benjamin Wolters, Timo Steffens

Real-world processes and modern multi-agent systems typically have high-dimensional characteristics that need to be configured and tuned for optimal performance. Optimizing processes and agent behaviors is an active field of research and relevant methods exist. This paper introduces a flexible and scalable framework that allows to plug in a variety of optimization methods. Two case studies demonstrate the flexibility of the framework.

1 Introduction

Learning and optimizing processes is an active field of research. Typically, such methods are evaluated and tuned on specific applications and implementations. However, typical applications for commercial use do not focus on the development of new methods, but apply existing methods in a broader application. The optimization method is only part of a larger suite which also provides functionalities to model, simulate, analyze and visualize processes (e.g. [4]). In such settings, it is necessary to easily integrate, configure and evaluate diverse optimization techniques.

This paper introduces a platform called LAMPSys, which is a general-purpose agent-based simulation system. It has been designed to satisfy requirements for operations research and process optimization applications. It applies the paradigm that agent-behaviors are processes that can be represented and executed in the same way as business processes. In this view, learning agent-behaviors and optimizing processes are equivalent. LAMPSys provides facilities to plug-in and evaluate process optimization techniques.

In order to illustrate the flexibility of LAMPSys, two case studies are presented, in which very different optimization techniques were integrated into the system.

The remainder of the paper is organized as follows. Section 2 outlines the simulation system, its architecture and the process modeling language that is used to specify the agent-behaviors. In section 3 two case-studies are described. Finally, the last section concludes and outlines future work.

2 LAMPSys

LAMPSys applies the Petri-net-based language LAMPS to represent processes and agent behaviors. LAMPS (Language for Agent-based Modeling of Processes and Scenarios). It extends the concept of colored Petri nets and hierarchical Petri nets [6]. The former introduced the concept of typed tokens, the latter structured so-called places (states) and actions hierarchically. The most important extension is the concept of agents that correspond to conditions in the Petri net model. Please refer to [4, 7] for more details.

LAMPSys is a simulation system that is able to execute LAMPS graphs. It enhances the Flip-Tick-Architecture (FTA) [5]. FTA is a design paradigm for scalable distributed systems

that exhibit priorly unknown dynamic characteristics as well as disturbances and inaccuracies which are difficult, if not impossible, to model in a closed-form mathematical approach.

LAMPSys is based on the concept of a society of agents. The whole environment is modeled in terms of agents and the whole interaction is based on effects of actions executed by agents. Additionally, agents can form so-called *clans*, which are headed by one agent acting as *clan-chief*. A clan can take certain functions for its clan members, e.g. scheduling or communication.

LAMPSys comprises three classes of entities: agents, clans, and tags. The society of all agents A is formed by a set of individual agents a_j : $A = \{a_1, \dots, a_j, \dots, a_k\}$. Each agent a_j is composed of a set of typed attributes U_j , whose value assignment determines the agent's state, together with an action function f_j , which entails all operations that can be performed by the agent $a_j = (U_j = (u_{1,j}, \dots, u_{m,j}), f_j)$.

The principle of autonomy of agents forbids the direct manipulation of internal data structures and behaviors of other agents. Consequently, all interactions between agents are handled via messages. Formally, a message N_v is comprised of a number of attributes: $N_v = \{u_{1,v}, \dots, u_{l,v}\}$. Upon receiving a request, the agent is able to analyze its content and to decide whether it wants to comply.

The basic unit of execution is called a cycle. During one cycle, the agent reads its messages and triggers the appropriate actions, which might consist of writing messages to other agent. This is called the *tick* of an agent.

During each tick a certain amount of time dt is simulated. Action executions that lasts longer than dt are split and partially executed several times in consecutive ticks. A scheduler-clan is a set of agents that have the same dt and, thus, simulate the world with the same pace. Time steps can be adapted from cycle to cycle and are determined by the scheduler-clan-chief. Thus, the time resolution of the simulation can vary from cycle to cycle. This is particularly valuable for increasing the time resolution in the computation of dynamic equations for fast moving objects. Note that different scheduler-clans can have different time resolutions. With this approach, fully synchronized (all agents form one scheduler-clan) as well as completely asynchronous systems (every agent forms its own scheduler-clan) can be modeled in terms of FTA.

The messages used for inter-agent communication are called tags. The agents can either communicate directly with other agents or they can register with one or more clan-chiefs of

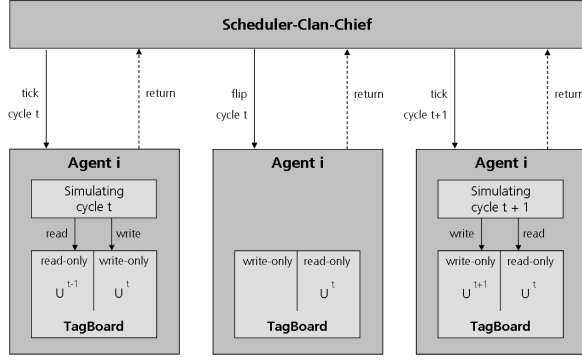


Figure 1: Scheduling and data storage with tag-board.

interaction-clans. Such an interaction-clan can set up any communication protocol, e.g. publish-and-subscribe or blackboard. These protocols are then managed by the clan-chief.

As mentioned above, the whole simulation is performed in cycles. Each agent manipulates data during the *ticking* which is stored in its so-called *tag-board*. It consists of two sides. One is write-only and contains all data written by the agent in cycle t . The other side is read-only and contains all data of cycle $t - 1$. After all agents of the corresponding scheduler-clan have been ticked, the tag-boards are *flipped* which causes the write-only side to become read-only containing the whole data valid at cycle t and the former read-side gets prepared to store the updates of the next cycle $t + 1$. Figure 1 sketches this idea. During tick t , the attribute values U^{t-1} are read and the values U^t are written. After the flipping, the agent reads the attribute values U^t and writes U^{t+1} .

3 Case studies

Two case studies are performed in the domain of medical evacuation, which is illustrated in the following section. A special agent, the *dispatcher agent* has the task to command the evacuation operation. Especially, it has to determine which injured is evacuated by which rescue transport. Section 3.2 describes a first greedy implementation of the dispatcher agent. Due to the flexibility of LAMPSys, it is very simple to provide several agents performing the same task. In section 3.3 a dispatcher agent is introduced, whose agent-behavior has been optimized by a genetic approach. The former dispatcher has been developed to estimate the quality of the latter one. Finally, section 3.4 concludes by comparing these two approaches.

3.1 Medical Evacuation Domain

The domain of medical evacuation is a logistics environment. At the beginning of the simulation some agents representing injured people are placed at random locations. The dispatcher agent can command some emergency transporters. Its task is to let these transporters rescue the injured by transporting them into hospitals. The transporters can either be helicopters or vans. All of them can have different average speeds and transport capacities. The most important difference between helicopters and vans is that the latter move on roads whereas the former

ones can fly the direct way. Another important parameter of the transporters is that they have different times to be activated and that they must be repaired after their usage. The injured have different degrees of injuries, ranging from light to severely wounded. Additionally, they have a certain amount of health points that are reduced over time. If all health points are lost, the agent perishes. The number of health points as well as the reduction rate depends on the degree of the injury. When the injured have been delivered at the hospital they can be healed, i. e. all heal points are restored.

The whole domain can be viewed as a logistics domain. The dispatcher has several transporters to deliver several packages to several waiting queues. An important point is that each package has a certain time limit for delivery, which must not be exceeded. The task is to deliver as many packages as possible. The whole problem can be modeled in terms of logistic chains. Each transporter is available at a certain location of the map (resource-area). Then it can be assigned a transport job consisting of several packages that must be collected and delivered at a waiting queue (working-area). Afterward it must be maintained (stand-by-area). Note that the stand-by-area of one dispatcher (the collect-injured-dispatcher) can be the working-area of another dispatcher (the repair-transporters-dispatcher). In that way arbitrary logistic chains can be hooked into each other and very complex problems can be solved based on that principle.

3.2 Greedy Approach

The greedy dispatcher uses the model of the logistic chain directly. It has the following parameters, which are enriched by agent types:

- *resource-areas*: Any transporter of the given type without assigned job in one of these polygons is regarded as a resource and can be assembled for evacuation.
- *collect-areas*: Any agent of the given type in these polygons must be evacuated.
- *deliver-areas*: This is a list of hospitals, which can be chosen as target for a transport job.
- *stand-by-areas*: This is a list of way points. Any point can be chosen as target to which the transporter of the given type moves after it has delivered its wounded.

The behavior of the greedy dispatcher agent is quite simple. At the beginning of each cycle, it checks all collect-areas for injured that have not yet been scheduled for evacuation. The collect-areas are processed in the order in which they occur in the parameters. If one is found, a free transporter (resource) is searched for. All resource-areas are processed in the modeled order. If a resource is found, it is assigned the transport job. Then again the collect-areas are searched and the current transport is ordered to collect the injured until its capacity is exhausted. Next the closest hospital is selected, whereas close means the euclidean distance from the last injured to the hospital's position. Afterward the first matching stand-by-area is selected. Finally the next injured is calculated and the process starts again. If a transporter has started its transportation it is not regarded as resource again until it has finished its job. Thus, it is not re-scheduled if new wounded occur. Note that the check of new injured is performed every cycle. This corresponds to a re-planning if new injured occur during the simulation.

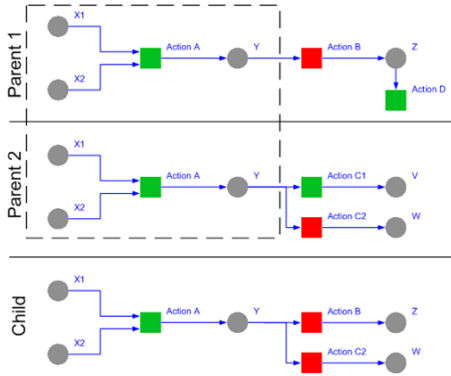


Figure 2: Example for a cross-over based on graph matching. The intersection of parent 1 and parent 2 is depicted as a dotted rectangle. The child behavior consists of the intersection of the two parents plus one randomly selected subgraph from each parent.

The performance of the dispatcher in terms of solution quality depends on the order of the collect, resource, and stand-by-areas only. Thus, it is easy to create situations in which the dispatcher behaves sub-optimal if the order of the areas is known.

3.3 Optimizing of agent-behaviors using a genetic algorithms

One method for process optimization is the usage of genetic algorithms. An agent-behavior can be built by a set of possible agent-actions that are combined to a LAMPS-graph. Thus, a behavior representing a logistic dispatching can also be built. An agent containing such a behavior is called *genetic dispatcher* in the following. In order to perform as optimal as possible, a sufficient range of training examples must be evaluated. In contrast to the greedy dispatcher, the genetic dispatcher does not use any domain knowledge. An external feedback is needed to estimate the quality of the dispatching behavior.

Genetic algorithms (GA) mimic the natural evolution [3]. The naming has also been influenced by biology. GA optimize a population of individuals, where each individual represents a possible solution to the optimization problem. A fitness function evaluates the solution quality of an individual. The evaluation steps of GA are called generations. For each generation, the so-called genetic operators are applied to the current population resulting in a new population, the next generation. Then the fitness function determines the fitness of each individual of the next generation. This step is crucial for the convergence of the algorithm. The process is iterated until a break condition holds. The algorithm usually starts with a randomly constructed population of solutions as initial generation. A brief overview over the genetic operators is given in the following. Please refer to [7] for further details.

Cross-over: This operator mimics the reproduction of two parent solutions into a new child solution. Conceptually, the child should contain traits or parts of both parents, while forming a new solution. Since LAMPS is based on Petri nets which can mathematically be described as graphs $G = (V, E)$, it is obvious to base the cross-over on a graph-matching algorithm.

	avg saved	dev saved	avg perfect	dev perfect
greedy	9.743	0.678	0.135	0.342
genetic	10.997	0.056	0.997	0.056

Table 1: Comparison of the greedy and genetic dispatcher. The average number of the evacuated injured (avg saved) and its standard deviation (dev saved) as well as the average number of perfect runs (avg perfect) and its standard deviation (dev perfect) have been calculated. A run is called perfect if all wounded have been rescued. 5000 runs have been performed for each dispatcher agent.

A graph-matching algorithm computes an intersection of two graphs. Conceptually, in the first step the intersection of both parent graphs is identified. If the intersection is empty, no child graph is produced and new parents are selected. The second phase extends the intersection graph by one sub-graph of each parent that is connected to the intersection but not part of it. If such a transaction does not exist, the intersection is not extended. Figure 3.3 shows an example of the cross-over operation. Note that the generated child must not violate the correct syntax of LAMPS graphs and should sufficiently differ from its parents.

Mutation: The mutation operator is applied randomly to some individuals and modifies them in a random fashion. It guarantees a wide range of genetic characteristics, i.e. agent-actions, during the learning process. Two basic mutations are performed on LAMPS graphs. Either an action is inserted or deleted. Thereby, the correct LAMPS syntax must not be violated.

Selection: The selection operator selects the individuals by their fitness that are transferred into the next generation. Additionally, it chooses a pair of parents, that are reproduced by the cross-over. In general, two random individuals are selected to produce two children. The selection operator is based on Stochastic Universal Sampling [1] which is a standard operator for selection. A fitness proportional selection guarantees a survival of more fitter solutions but also transfers some less fit solutions to the next generation. Additionally the operator selects pairs of individuals for the cross-over. In every case, the complete generation will be exchanged by the new selected individuals. This is a good strategy to prevent learning a local optimum [2].

3.4 Evaluation

In order to compare the greedy and genetic logistic dispatcher, we set up a test scenario. At the beginning of the simulation, a set of injured people is distributed uniformly over a specified region with the size of a medium-city (with a population of approx. 350.000). The set of injured consists of 3 heavily, 3 medium and 5 light wounded. The dispatcher can use one helicopter and one van to evacuate the wounded. Each transporter has a transport capacity of one. The scenario finishes when all injured have been evacuated or perished. A run is called perfect if all wounded are healed.

Table 1 shows the results of the comparison. The genetic dispatcher outperformed the greedy one. We compared two measures of effectiveness. First, we calculated the number of

rescued injured during 5000 runs for each dispatcher. Since during each run 11 wounded occur, the overall number of injured is 55000. Both dispatchers were able to rescue most of them. However, the genetic dispatcher performed superior and healed more than 99 percent of the wounded. Additionally, we calculated the number of perfect runs. A perfect run occurs if all injured are rescued, i.e. the dispatcher performed optimally. This measure really points out the superiority of the genetic dispatcher, which performed in more than 99 percent of its runs perfectly. The greedy dispatcher only reached a rate of 13 percent of optimal runs. In both measures of effectiveness, the genetic dispatcher performed significantly better than the greedy one in terms of a two-tailed t-test.

Thus, the genetic algorithm optimization resulted in a logistic dispatcher that outperformed the hand-coded greedy dispatcher. Since a genetic algorithm does not use any domain knowledge, arbitrary behaviors and processes can be optimized automatically. Note that during this case study, the behavior was trained on some training data, but worked well in 5000 different settings of this medical evacuation scenario.

4 Conclusion

LAMPSSys' important concept of autonomous agents and the idea to model the whole domain in terms of agents, enables LAMPSSys to use only one interface (the agent-interface), which has to be implemented in order to incorporate a new technique. Thus, the system is very flexible and modular. Many different approaches can be combined to optimize and model interesting domains. As case studies, we introduced two methods of our framework and evaluated them in the domain of medical evacuation. The automatically optimized dispatcher outperformed the hand-modeled one significantly.

In the future we are going to enrich LAMPSSys by several techniques to improve autonomous decision making. On the one hand, we aim to integrate state-of-the-art planning systems like PDDL planner, HTN planning systems etc. On the other hand, we want to extend and formalize the concept of commander and subordinate in order to form a hierarchical functional control structure. Then it is even more simple to integrate different optimization techniques. This aspect is also interesting since we are already using the clan concept on the technical level, but not on the functional one. Another research direction is the monitoring of distributed plan execution.

References

- [1] James E. Baker. Reducing bias and inefficiency in the selection algorithm. In *Proceedings of the Second International Conference on Genetic Algorithms and their application*, pages 14–21, Mahwah, NJ, USA, 1987. Lawrence Erlbaum Associates, Inc.
- [2] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2 edition, 1989.
- [3] John H. Holland. *Adaptation in natural and artificial systems*. MIT Press, Cambridge, MA, USA, 1 edition, 1992.
- [4] Philipp Huegelmeyer, Timo Steffens, and Thomas Zoeller. Specifying and simulating modern warfare scenarios with ITSimBw. In L. F. Perrone, F. P. Wieland, J. Liu, B. G. Lawson and D. M.

Nicol, and R. M. Fujimoto, editors, *Proceedings of the 2006 Winter Simulation Conference*. ACM, 2006.

- [5] Gerhard Richter. Flip-tick architecture: A cycle-oriented architecture for distributed problem solving. Technical report, GMD, Sankt Augustin, Germany, 1999. GMD Report No. 19.
- [6] Tomas Vojnar. Various kinds of petri nets in simulation and modelling. In J. Stefan, editor, *Proceedings of 31st Spring International Conference on Modelling and Simulation of Systems MOSIS '97*, volume 1, pages 227–232, 1997.
- [7] Benjamin Wolters and Timo Steffens. Learning agent-behavior for agent-based simulation using genetic algorithms. In *Proceedings of the European Simulation and Modeling Conference 2008*, pages 284–288. IEEE Computer Society Press, 2008.

Contact

Dipl.-Inform. Christoph Mies
 Dipl.-Inform. Benjamin Wolters
 Fraunhofer IAIS
 Schloß Birlinghoven, 53754 Sankt Augustin
christoph.mies@iais.fraunhofer.de
benjamin.wolters@iais.fraunhofer.de
<http://www.iais.fraunhofer.de/>

Bild

Christoph Mies studied Computer Science at Aachen University and received his Diploma in 2007. Afterward he got a research position at the Fraunhofer Institute IAIS with the focus on multi-agent systems in simulation and robotics.

Bild

Benjamin Wolters received his Diploma in Computer Science from Cologne University of Applied Science in 2008. During his study and his further work at the Fraunhofer Institute IAIS as a research assistant, he focused his work on multi-agent simulations and optimizing of autonomous processes.

Bild

Timo Steffens studied Artificial Intelligence and Computational Linguistics at the University of Osnabrueck. He received his PhD in Cognitive Science at the same university in 2006. Since 2005 he is a researcher at the Fraunhofer IAIS, where his research interests include multi-agent systems and machine learning.